Entity resolution, step by step

This notebook runs the pipeline that `scripts/resolve.py` runs on a file of any size: the same `er.run.Run`, one step per section, on a small generated population (about 3,700 records) so that every step can be seen. It holds no logic of its own; each section calls the code, then charts what the step did. Every code cell prints its Big-O, filled in with this file's numbers, and its time. The whole notebook runs in about half a minute.

The run never reads the truth (which records are one entity). It exists only because the data is generated, and the notebook reads it to show what each step got right.

1 Setup · 2 Generate · 3 Standardize · 4 Rules · 5 Candidates · 6 Fit · 7 Score · 8 Why a pair matches · 9 Threshold · 10 Cluster and evaluate · 11 Increment · 12 How the configuration was chosen · 13 Cost as the file grows

N records, L characters per value, P pairs inside a key's blocks, C candidate pairs, K comparisons, S sampled pairs.

1 · Setup

The chosen pipeline is `config/final.toml`, which `scripts/choose.py` wrote. The demo's population is `config/generator.toml`'s, made smaller. Charts use one colour per meaning: blue for the same entity (or evidence for a match), orange for different entities, red for evidence against.

In [1]:
```
import logging
import math
import shutil
import time
from contextlib import contextmanager
from dataclasses import replace

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.ticker import FuncFormatter, PercentFormatter
```

```
from matplotlib_inline.backend_inline import set_matplotlib_formats

from er.candidates import found_by
from er.config import column_name, load_pipeline
from er.evaluate import cluster_metrics, missed_pairs, pair_metrics, predicted_pairs
from er.generate import check, generate, load_reference, load_spec
from er.io import table_after
from er.match import SCORED_U, parameters, scaled_prior, waterfall
from er.resolve import connected_components
from er.run import Run
from er.settings import load_settings

logging.disable(logging.WARNING)
S = load_settings()
FINAL = load_pipeline(S.root / "config" / "final.toml", S.standardize, S.chains, S.columns)
SPEC = replace(load_spec(S.root / "config" / "generator.toml")[0], entities=3_000)  # the demo's size
DEMO = S.paths["artifacts"] / "demo"
DEMO.mkdir(parents=True, exist_ok=True)
UID, ENT = S.data.unique_id, S.data.entity_id
L, R = f"{UID}_l", f"{UID}_r"
FIELDS = [f for f in S.columns if f not in ("id", "entity")]  # the logical fields of config/settings.toml [column
s]
pd.set_option("display.max_columns", 30, "display.width", 200)

BLUE, ORANGE, RED, INK, INK2, MUTED, GRID, AXIS, SURFACE = ("#2a78d6", "#eb6834", "#e34948", "#0b0b0b", "#52514e",
                                                            "#898781", "#e1e0d9", "#c3c2b7", "#fcfcfb")
set_matplotlib_formats("svg")
plt.rcParams.update({"figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "savefig.facecolor": SURFACE,
                     "axes.edgecolor": AXIS, "axes.linewidth": 0.8, "axes.labelcolor": INK2, "axes.titlecolor": IN
K,
                     "axes.titlesize": 10, "axes.titleweight": "bold", "axes.titlelocation": "left",
                     "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "axes.axisbelow": Tr
ue,
                     "grid.color": GRID, "grid.linewidth": 0.6, "xtick.color": MUTED, "ytick.color": MUTED,
                     "xtick.labelcolor": INK2, "ytick.labelcolor": INK2, "font.size": 9, "lines.linewidth": 2,
                     "legend.frameon": False, "figure.dpi": 100})


@contextmanager
def timed(big_o: str):  # a cell's own work, outside the run's steps
    t = time.perf_counter()
```

```
    yield
    print(f"{big_o} · {time.perf_counter() - t:.2f}s")


def done(run: Run) -> None:  # the step just run: its Big-O with this file's numbers, and its time
    s = run.steps[-1]
    print(f"{s['step']}: {s['big_o']} = {s['filled']} · {s['seconds']:.2f}s")


def same(pairs: pd.DataFrame) -> np.ndarray:  # the truth, read only to show what a step got right
    return pairs[L].map(entity).to_numpy() == pairs[R].map(entity).to_numpy()


def finish(fig) -> None:
    fig.tight_layout()
    plt.show()
```

2 · Generate

A synthetic population built from public reference data: each entity, a person or a company, is written as one or more records with typos, empty fields and life changes (a move, a new phone, a married surname). The truth is kept apart. Below: the entity whose three records differ the most, then how many records each entity has and how often each column is filled. O(E·r) for E entities of r records each.

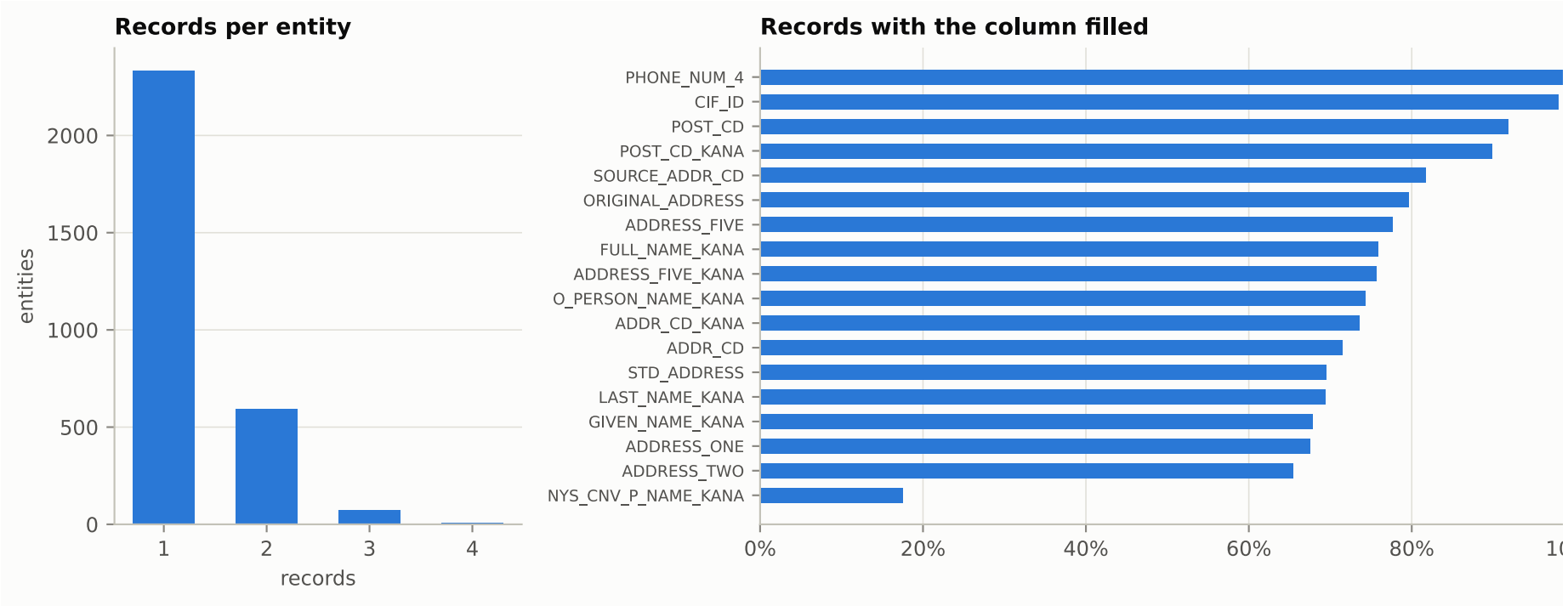
In [2]:
```
with timed(f"generate: O(E·r) = O({SPEC.entities:,} entities · their records)"):
    g = generate(SPEC, *load_reference(S.paths["reference"]))
    check(SPEC, g)
g.df.to_parquet(DEMO / "records.parquet", index=False)
entity = g.truth.set_index(UID)[ENT]
per_entity = entity.value_counts()
print(f"{len(g.df):,} records · {len(per_entity):,} entities · {(per_entity > 1).mean():.0%} with more than one re
cord")

raw = g.df.set_index(UID)
varied = raw.groupby(entity.reindex(raw.index).to_numpy()).nunique().gt(1).sum(axis=1)  # columns that differ, per
entity
example = varied[per_entity.reindex(varied.index).eq(3)].idxmax()
ex_ids = entity.index[entity == example]
display(raw.loc[ex_ids].dropna(axis=1, how="all"))
```

```
fig, (a, b) = plt.subplots(1, 2, figsize=(10, 3.8), width_ratios=[1, 2])
counts = per_entity.value_counts().sort_index()
a.bar(counts.index.astype(str), counts.to_numpy(), width=0.6, color=BLUE)
a.set(title="Records per entity", xlabel="records", ylabel="entities")
a.grid(axis="x", visible=False)
fill = raw.notna().mean().sort_values()
b.barh(fill.index, fill.to_numpy(), height=0.6, color=BLUE)
b.set(title="Records with the column filled", xlim=(0, 1))
b.grid(axis="y", visible=False)
b.xaxis.set_major_formatter(PercentFormatter(1))
b.tick_params(axis="y", labelsize=7)
finish(fig)
```

```
generate: O(E·r) = O(3,000 entities · their records) · 0.62s
3,747 records · 3,000 entities · 22% with more than one record
```

CONT_ID	PHONE_NUM_4	CIF_ID	FULL_NAME_KANA	O_PERSON_NAME_KANA	POST_CD	POST_CD_KANA	LAST_NAME_KANA	GIVEN_NAME_KANA	SOURCE_ADDR_CD	ORIGINAL_ADDRESS	ADDRESS_FIVE	ADDRESS_FIVE_KANA	ADDR_CD_KANA	ADDR_CD	STD_ADDRESS	ADDRESS_ONE	ADDRESS_TWO
0000000276	2769	2646970416577	ヨシダアガリ	ヨシダ　アガリ	2310865	2310865	ヨシダ	アガリ	304949033 8	<NA>	<NA>	1-19-11	<NA>	14104006001	<NA>	神奈川県	横浜市中区
0000000796	2769	2646970416577	マツナガアカリ	<NA>	2240029	2240029	<NA>	アカリ	7485606977	神奈川県横浜市都筑区南山田3丁目9番28号	3-9-28	3-9-28	14118003003	14118003003	神奈川県横浜市都筑区南山田3-9-28	神奈川県	横浜市都筑区
0000003512	2769	1225159189230	マツナガアカリ	マツナガ　アカリ	2310865	2310865	マツナガ	アカリ	304949033 8	神奈川県横浜市中区北方町1丁目19番11号	1-19-11	1-19-11	14104006001	14104006001	神奈川県横浜市中区北方町1-19-11	神奈川県	横浜市中区



3 · Standardize

`load` reads the file in chunks and checks it against `config/pipeline.toml` `[fields]`: the id is mandatory, and at least one of the person and company names is needed. An optional field the file lacks disables only what reads it. It then builds each logical field from `config/settings.toml` `[columns]` (the first filled source wins) and the normalizations the pipeline reads: kana folding, legal forms dropped, addresses cut to the street number. Each record is standardized on its own, so chunks change nothing. O(N·L).

In [3]:
```
run = Run(FINAL, S)
run.load(DEMO / "records.parquet")
done(run)
print("disabled:", ", ".join(run.disabled) or "nothing: every optional field is in the file")
std = run.con.sql("SELECT * FROM records").df().set_index(UID)

display(std.loc[ex_ids, FIELDS])  # the entity above, in logical fields
rows = []
for field, chain in [("person_name", "fold"), ("company_name", "company"), ("address", "address")]:
```

```
    after = column_name(field, S.chains[chain])
    changed = std.loc[std[field].notna() & (std[field] != std[after]), [field, after]].head(3)
    rows.append(changed.set_axis(["before", "after"], axis=1).assign(field=field, normalization=" → ".join(S.chain
s[chain])))
display(pd.concat(rows)[["field", "normalization", "before", "after"]])
```

```
load: O(N·L) = O(3,747·L) · 0.04s
disabled: nothing: every optional field is in the file
```

CONT_ID	full_name	person_name	given_name	company_name	any_name	address	address_code	postal_code	prefecture	phone	customer_id
0000000276	ヨシダアガリ	ヨシダアガリ	アガリ	NaN	ヨシタアカリ	NaN	3049490338	2310865	神奈川県	2769	2646970416577
0000000796	マツナガアカリ	マツナガアカリ	アカリ	NaN	マツナカアカリ	神奈川県横浜市都筑区南山田3-9-28	7485606977	2240029	神奈川県	2769	2646970416577
0000003512	マツナガアカリ	マツナガアカリ	アカリ	NaN	マツナカアカリ	神奈川県横浜市中区北方町1-19-11	3049490338	2310865	神奈川県	2769	1225159189230

CONT_ID	field	normalization	before	after
0000000001	person_name	kana_fold	マミヤアギ	マミヤアキ
0000000004	person_name	kana_fold	ニシムラガサオト	ニシムラカサオト
0000000007	person_name	kana_fold	ワタナベスズハ	ワタナヘススハ
0000000002	company_name	nfkc → strip_legal_form → kana_fold	ジョウコウカブシキガイシャ	シヨウコウ
0000000011	company_name	nfkc → strip_legal_form → kana_fold	アンゼンエイラクコウツウカブシキガイシャ	アンセンエイラクコウツウ
0000000013	company_name	nfkc → strip_legal_form → kana_fold	カブシキガイシャバイカドウ	ハイカトウ
0000000001	address	address	秋田県北秋田市阿仁小様20-26	北秋田市阿仁小様20-26
0000000002	address	address	京都府京都市左京区下鴨水口町18-6	京都市左京区下鴨水口町18-6
0000000003	address	address	山口県山口市大内矢田7番25号	山口市大内矢田7-25

4 · Rules

Stage 1 is deterministic. An exact rule joins records equal on all its keys; a fuzzy rule needs some keys equal and others close (names within two edits, company names by shared characters, addresses by Jaro–Winkler, a phone one digit off). Before any join, `budget` counts every rule's pairs P = Σ C(k, 2) over its key blocks and stops the run if one is over budget. The first rule in order wins a pair, and a fuzzy pair is kept only if it touches a record the exact rules left alone. Left: how many true pairs the rules join, clusters included. Right: the true pairs each rule joins itself.

In [4]:

```
plan = run.budget()
done(run)
run.rules()
done(run)
kept = run.con.sql("SELECT * FROM rule_kept").df().assign(true=same)
order = [r.name for st in FINAL.rules.stages for r in st.rules]
stage_of = {r.name: st.name for st in FINAL.rules.stages for r in st.rules}
per_rule = (plan[plan["kind"] == "rule"].set_index("name")[["est_pairs", "largest_block"]].rename(columns={"est_pa
irs": "P"})
            .join(kept.groupby("rule")["true"].agg(joined="size", true="sum")).fillna(0).astype(int).reindex(orde
r))
display(per_rule.rename_axis("rule"))

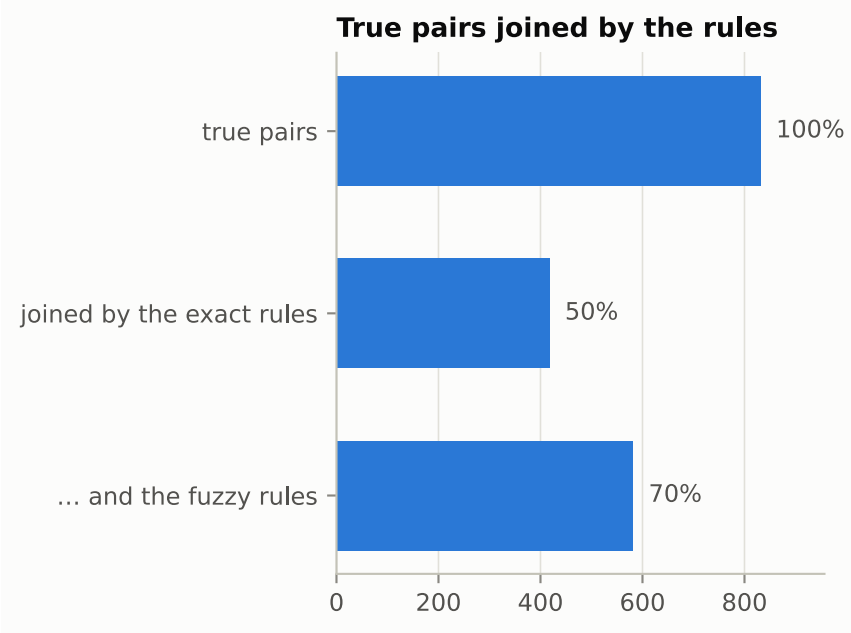
rule_clusters = run.con.sql("SELECT * FROM rule_clusters").df().set_index(UID)["cluster_id"]
exact_only = connected_components(pd.Series(std.index), kept[kept["stage"] == "exact"], run.db, UID)
m_exact, m_rules = (pair_metrics(c, entity.reindex(c.index)) for c in (exact_only, rule_clusters))
total = m_rules["TP"] + m_rules["FN"]
print(f"the rules join {m_rules['recall']:.1%} of the {total:,} true pairs, with {m_rules['FP']} false")

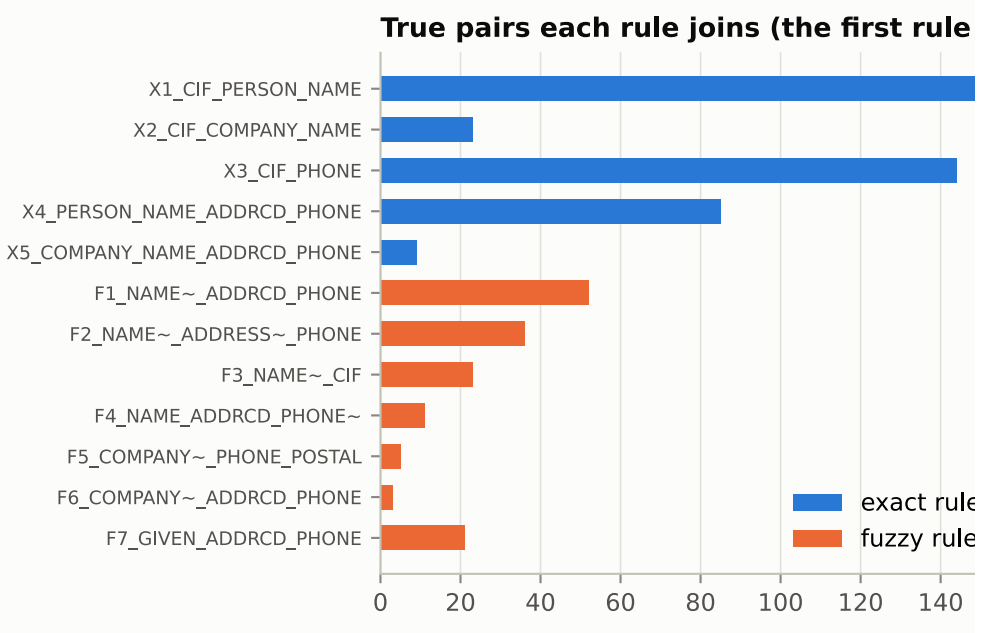
fig, (a, b) = plt.subplots(1, 2, figsize=(10, 3.4), width_ratios=[1, 1.3])
funnel = pd.Series({"true pairs": total, "joined by the exact rules": m_exact["TP"], "… and the fuzzy rules": m_ru
les["TP"]})[::-1]
a.barh(funnel.index, funnel.to_numpy(), height=0.6, color=BLUE)
for y, v in enumerate(funnel.to_numpy()):
    a.text(v, y, f"  {v / total:.0%}", va="center", color=INK2)
a.set(title="True pairs joined by the rules", xlim=(0, total * 1.15))
a.grid(axis="y", visible=False)
y = np.arange(len(order))[::-1]
for stage, colour in (("exact", BLUE), ("fuzzy", ORANGE)):
    mask = np.array([stage_of[r] == stage for r in order])
    b.barh(y[mask], per_rule["true"].to_numpy()[mask], height=0.6, color=colour, label=f"{stage} rules")
b.set_yticks(y, order, fontsize=7)
b.set(title="True pairs each rule joins (the first rule wins a pair)")
b.grid(axis="y", visible=False)
b.legend(loc="lower right")
finish(fig)
```

```
budget: O(N) = O(3,747) per rule and generator · 0.03s
rules: O(N + Σ P_r·L²) + O((N + E)·d) = O(3,747 + 564 pairs) · 0.23s
```

rule	P	largest_block	joined	true
X1_CIF_PERSON_NAME	151	3	151	151
X2_CIF_COMPANY_NAME	23	3	23	23
X3_CIF_PHONE	275	3	144	144
X4_PERSON_NAME_ADDRCD_PHONE	164	3	85	85
X5_COMPANY_NAME_ADDRCD_PHONE	18	3	9	9
F1_NAME~_ADDRCD_PHONE	362	4	52	52
F2_NAME~_ADDRESS~_PHONE	391	4	36	36
F3_NAME~_CIF	366	3	23	23
F4_NAME_ADDRCD_PHONE~	217	3	11	11
F5_COMPANY~_PHONE_POSTAL	433	4	5	5
F6_COMPANY~_ADDRCD_PHONE	362	4	3	3
F7_GIVEN_ADDRCD_PHONE	175	3	21	21

```
the rules join 70.0% of the 832 true pairs, with 0 false
```





5 · Candidates

Stage 2 compares only pairs that some generator lists. A key generator pairs records equal on all its keys; a deletion-key generator pairs names one edit apart (each value and its one-character deletions, within a postal area); a sorted window pairs neighbours in sorted order. Each is one SQL join or window over the records table; their pairs are merged, and pairs inside one rule cluster are dropped. Every key here pairs a share of the records that stays flat as the file grows: a field with a fixed pool of values (the phone's last digits, an address shared by a household) is always combined with another. O(N·L + Σ P_g).

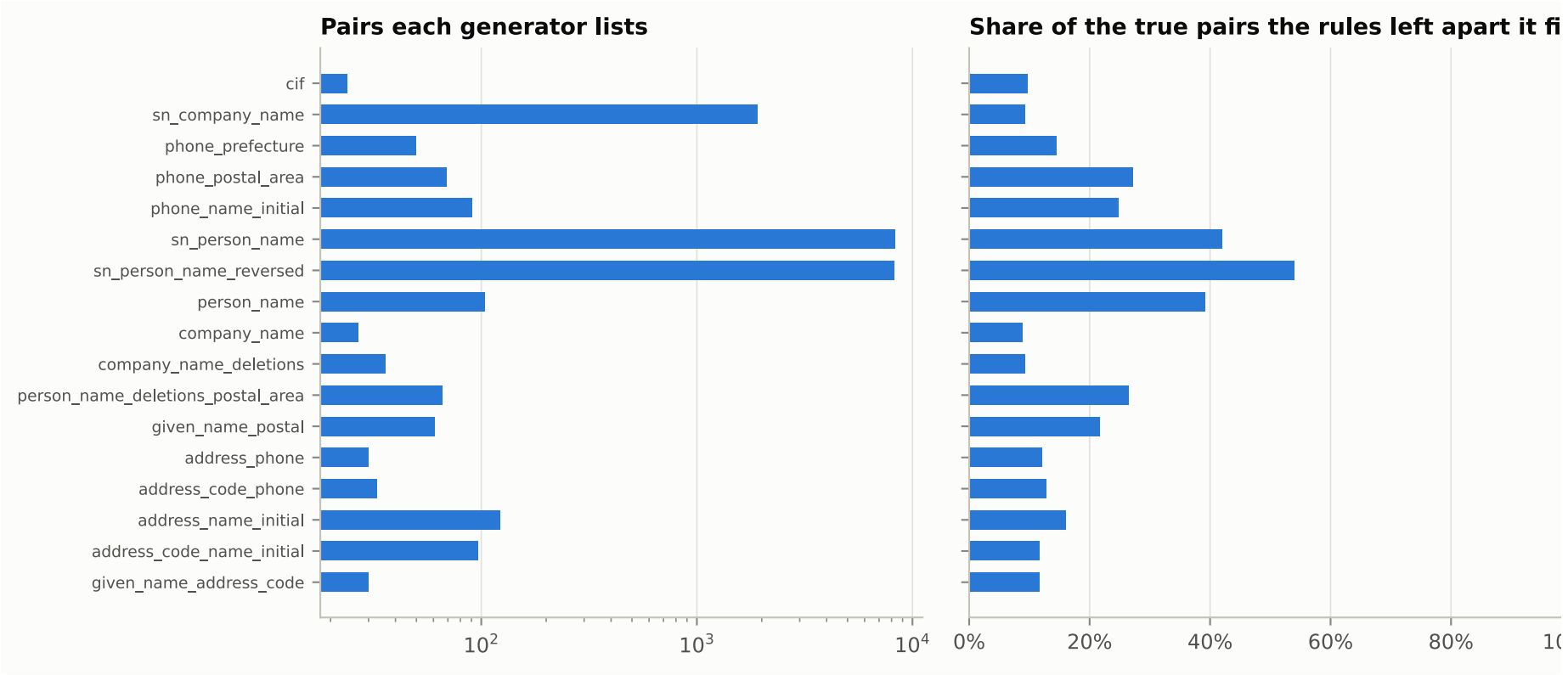
In [5]:
```
report = run.candidates()
done(run)
cands = run.con.sql("SELECT * FROM cands").df().assign(true=same)
targets = missed_pairs(rule_clusters, entity.reindex(rule_clusters.index), UID)  # true pairs the rules left apart
wanted = set(zip(targets[L].astype(str), targets[R].astype(str)))
on_target = np.array([k in wanted for k in zip(cands[L], cands[R])])
report["finds"] = [(found_by(cands, {name}) & on_target).sum() / len(wanted) for name in report["generator"]]
report["different_entity_pairs"] = [(found_by(cands, {name}) & ~cands["true"]).sum() for name in report["generato
r"]]
display(report[["generator", "big_o", "pairs", "finds", "different_entity_pairs", "seconds"]].round(3))
print(f"{len(cands):,} candidate pairs · together they list {on_target.sum() / len(wanted):.1%} of the {len(wante
d):,} true pairs the rules left apart")

fig, (a, b) = plt.subplots(1, 2, figsize=(10, 4.2), sharey=True)
y = np.arange(len(report))[::-1]
a.barh(y, report["pairs"], height=0.6, color=BLUE)
a.set(title="Pairs each generator lists", xscale="log")
b.barh(y, report["finds"], height=0.6, color=BLUE)
b.set(title="Share of the true pairs the rules left apart it finds", xlim=(0, 1))
b.xaxis.set_major_formatter(PercentFormatter(1))
a.set_yticks(y, report["generator"], fontsize=7)
for ax in (a, b):
    ax.grid(axis="y", visible=False)
finish(fig)
```

```
candidates: O(N·L + Σ P_g) = O(3,747·L + 19,357) · 0.06s
```

	generator	big_o	pairs	finds	different_entity_pairs	seconds
0	cif	O(N + P)	24	0.096	0	0.001
1	sn_company_name	O(N log N + N·w)	1914	0.092	1891	0.002
2	phone_prefecture	O(N + P)	50	0.144	14	0.001
3	phone_postal_area	O(N + P)	69	0.272	1	0.001
4	phone_name_initial	O(N + P)	91	0.248	29	0.001
5	sn_person_name	O(N log N + N·w)	8319	0.420	8214	0.002
6	sn_person_name_reversed	O(N log N + N·w)	8284	0.540	8149	0.002
7	person_name	O(N + P)	104	0.392	6	0.001
8	company_name	O(N + P)	27	0.088	5	0.001
9	company_name_deletions	O(N·L + P)	36	0.092	13	0.003
10	person_name_deletions_postal_area	O(N·L + P)	66	0.264	0	0.006
11	given_name_postal	O(N + P)	61	0.216	7	0.001
12	address_phone	O(N + P)	30	0.120	0	0.001
13	address_code_phone	O(N + P)	33	0.128	1	0.001
14	address_name_initial	O(N + P)	122	0.160	82	0.001
15	address_code_name_initial	O(N + P)	97	0.116	68	0.001
16	given_name_address_code	O(N + P)	30	0.116	1	0.001

18,587 candidate pairs · together they list 88.0% of the 250 true pairs the rules left apart



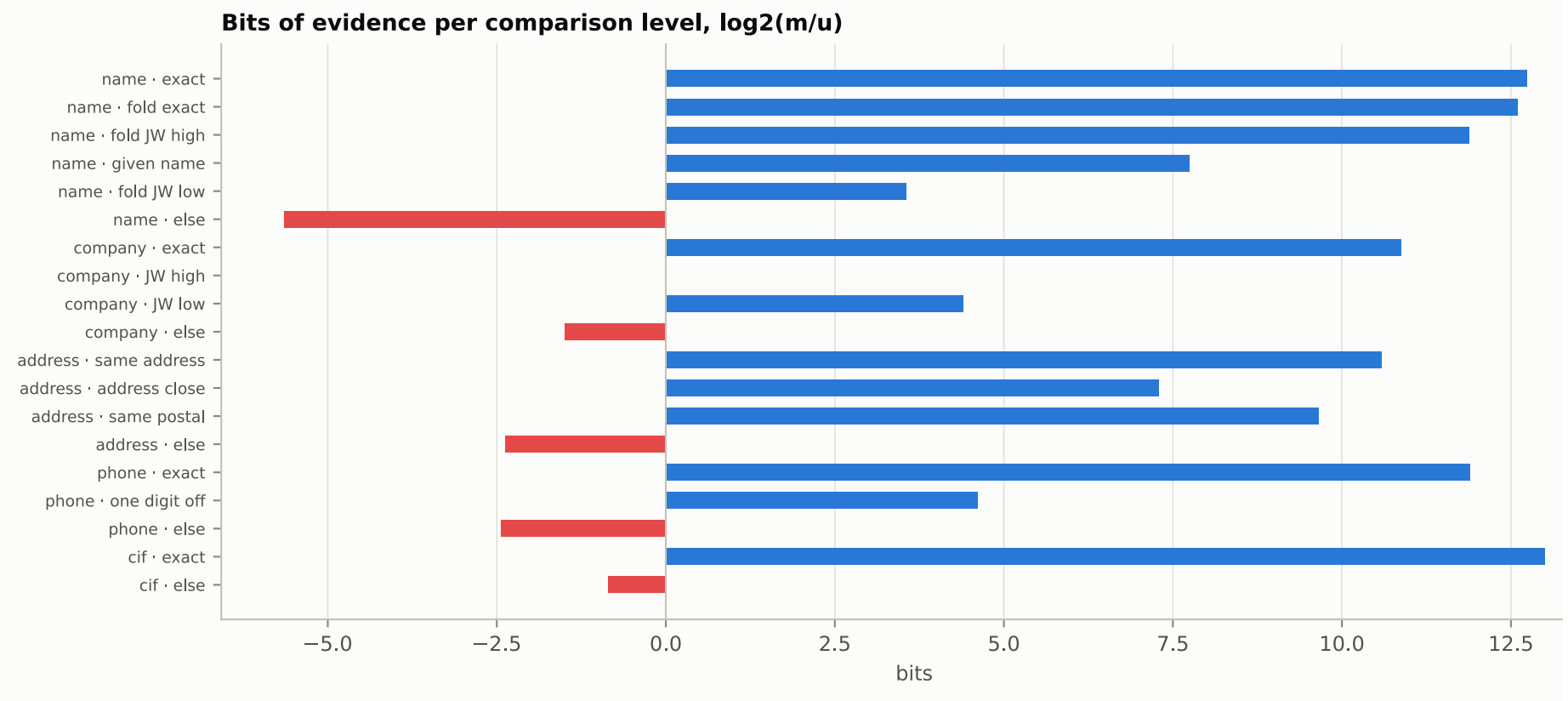
6 · Fit, without labels

Fellegi–Sunter weighs each comparison's agreement level by how much likelier it is for a true match than by chance: log2(m/u) bits. With no labels, λ (the chance two random records match) comes from near-certain rules and their measured recall; u (agreement by chance) from value counts for each comparison's leading exact levels and a random sample of pairs for the rest; m (agreement for true matches) by EM on blocks of equal phones or addresses. The fit uses a fixed-size sample of whole postal areas, so its cost doesn't grow with the file. O(n + S·K + P_em·K).

In [6]:
```
model = run.fit()
done(run)
lam = scaled_prior(model, run.n)
print(f"λ = {lam:.2e} at {run.n:,} records: a pair starts at {math.log2(lam / (1 - lam)):.1f} bits")
params = parameters(model).dropna(subset=["weight"]).iloc[::-1]
```

```
fig, ax = plt.subplots(figsize=(10, 4.4))
ax.barh(params["comparison"] + " · " + params["level"], params["weight"], height=0.6,
        color=np.where(params["weight"] >= 0, BLUE, RED))
ax.axvline(0, color=AXIS, linewidth=0.8)
ax.set(title="Bits of evidence per comparison level, log2(m/u)", xlabel="bits")
ax.grid(axis="y", visible=False)
ax.tick_params(axis="y", labelsize=7)
finish(fig)
```

```
fit: O(n + S·K + P_em·K) = n = 3,747 · 6.24s
λ = 7.38e-05 at 3,747 records: a pair starts at −13.7 bits
```



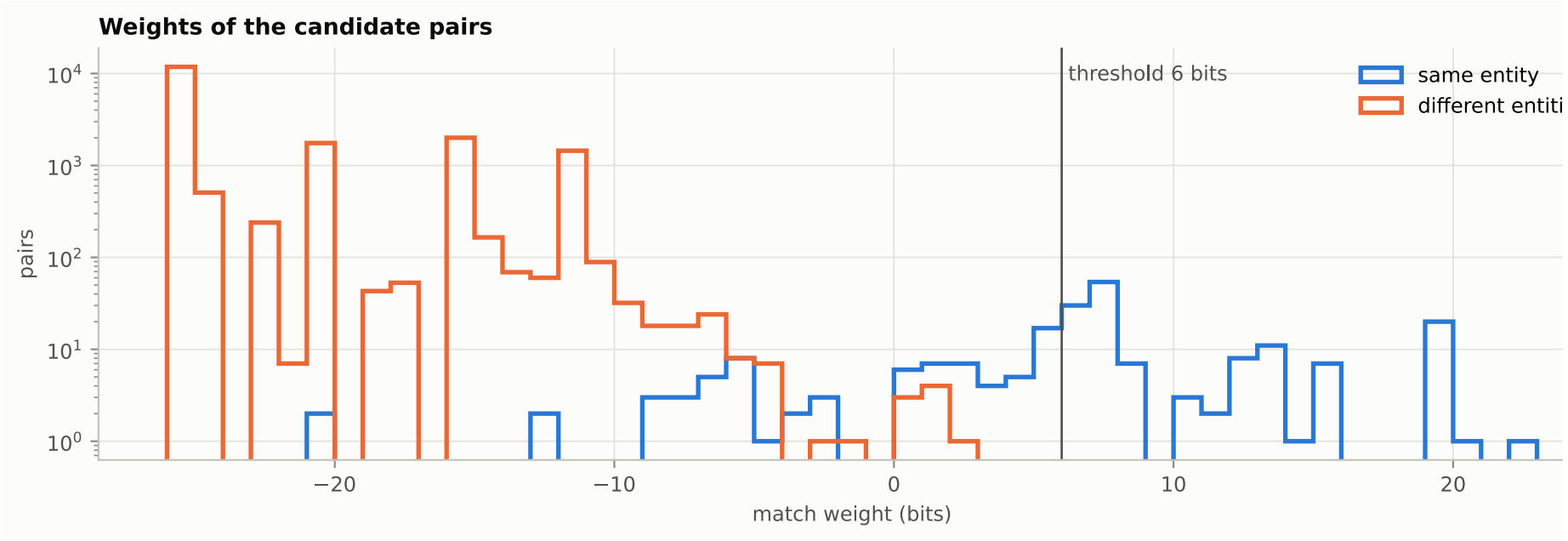
7 · Score

u is measured again on the file being scored: how often two records agree by chance depends on how many records there are, and u from a smaller file would understate the evidence of an agreement on a bigger one. Then every candidate pair gets its weight: the prior plus each comparison's bits. A pair is accepted at the threshold `scripts/choose.py` chose. O(N + S·K) + O(C·K·L²).

In [7]:
```
run.score()
done(run)
scores = run.con.sql("SELECT * FROM scores").df().assign(same=same)
threshold = run.pipeline.fs.min_match_weight
w = scores["match_weight"]
print(f"{len(scores):,} pairs scored · {(w >= threshold).sum():,} reach {threshold:g} bits, "
      f"{(scores['same'] & (w >= threshold)).sum():,} of them true")

fig, ax = plt.subplots(figsize=(10, 3.4))
bins = np.arange(math.floor(w.min()), math.ceil(w.max()) + 1)
ax.hist(w[scores["same"]], bins, histtype="step", color=BLUE, linewidth=2, label="same entity")
ax.hist(w[~scores["same"]], bins, histtype="step", color=ORANGE, linewidth=2, label="different entities")
ax.axvline(threshold, color=INK2, linewidth=1)
ax.text(threshold, ax.get_ylim()[1], f" threshold {threshold:g} bits", va="top", color=INK2)
ax.set(title="Weights of the candidate pairs", xlabel="match weight (bits)", ylabel="pairs", yscale="log")
ax.legend(loc="upper right")
finish(fig)
```

```
score: O(N + S·K) + O(C·K·L²) = O(3,747 + 10,000,000·5) + O(18,587·5·L²) · 5.66s
18,587 pairs scored · 145 reach 6 bits, 145 of them true
```



8 · Why a pair matches

A weight is a sum, so it can be read term by term. Two pairs: the accepted true match closest to the threshold whose names differ, and the rejected pair of different entities closest to it. Each bar is one term: the prior, then the bits of the level each comparison landed in.

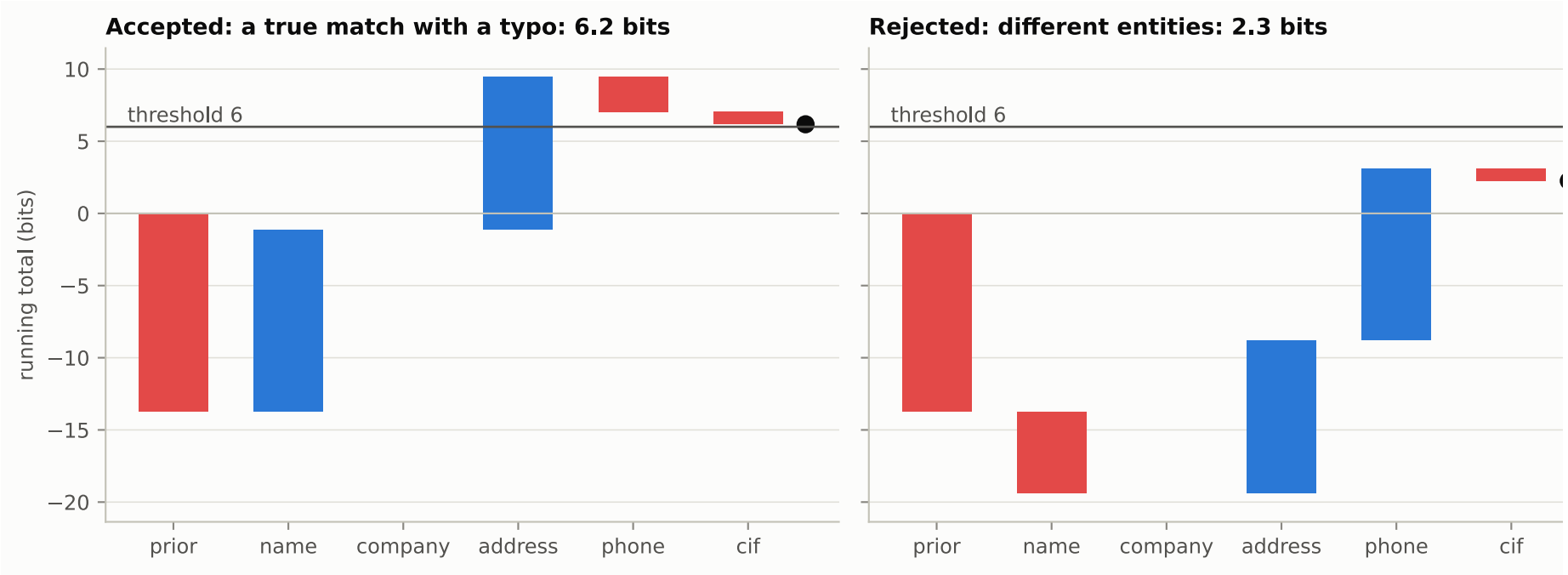
In [8]:
```
show = list(FINAL.explain_show)
filled = list(FINAL.explain_filled)
left, right = std.loc[scores[L], show].set_index(scores.index), std.loc[scores[R], show].set_index(scores.index)
both_filled = (left[filled].notna() & right[filled].notna()).all(axis=1)
differs = (left[filled[0]] != right[filled[0]]).fillna(False)
accepted = w >= threshold
picked = {"Accepted: a true match with a typo": scores[accepted & scores["same"] & both_filled & differs].nsmalles
t(1, "match_weight"),
          "Rejected: different entities": scores[~accepted & ~scores["same"]].nlargest(1, "match_weight")}

fig, axes = plt.subplots(1, 2, figsize=(10, 3.6), sharey=True)
ends = []
for ax, (title, pair) in zip(axes, picked.items()):
    row = pair.iloc[0]
```

```
    display(std.loc[[row[L], row[R]], show])
    bits = waterfall(run.scoring, {c.name: row[f"gamma_{c.name}"] for c in run.pipeline.fs.model.comparisons}, ru
n.n)
    end = bits["bits"].cumsum().to_numpy()
    start = np.concatenate([[0], end[:-1]])
    ends += [*start, *end]
    ax.bar(bits["evidence"], bits["bits"], bottom=start, width=0.6, color=np.where(bits["bits"] >= 0, BLUE, RED))
    ax.plot(len(bits) - 0.5, end[-1], marker="o", markersize=7, color=INK, clip_on=False)
    ax.axhline(threshold, color=INK2, linewidth=1)
    ax.text(-0.4, threshold, f"threshold {threshold:g}", va="bottom", color=INK2)
    ax.axhline(0, color=AXIS, linewidth=0.8)
    ax.set(title=f"{title}: {row['match_weight']:.1f} bits")
    ax.grid(axis="x", visible=False)
axes[0].set(ylabel="running total (bits)", ylim=(min(ends) - 2, max(ends + [threshold]) + 2))
finish(fig)
```

CONT_ID	full_name	company_name	address	phone	customer_id
0000001124	イジワタソウヘイ	NaN	岐阜県多治見市市之倉町9丁目17番15号	0596	2191723268496
0000001621	イシワタソウヘイ	NaN	岐阜県多治見市市之倉町9-17-15	7053	6560287423333

CONT_ID	full_name	company_name	address	phone	customer_id
0000001422	NaN	NaN	宮城県白石市鷹巣1-8　グランドール407	7123	7519543708544
0000003170	ホソカワカノオウ	NaN	宮城県白石市鷹巣1-6　コーポ山田807	7123	1724974176586



9 · Threshold

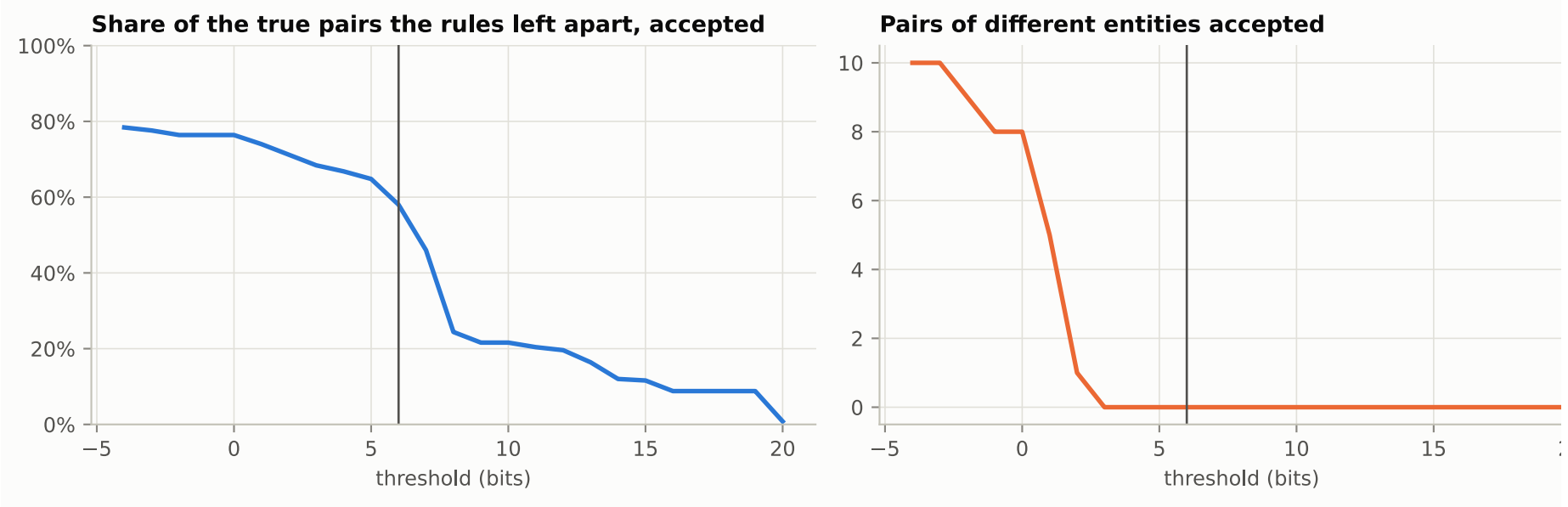
`scripts/choose.py` chose the threshold on cross-validated labelled data: the lowest weight at which no false pair was accepted and the upper bound of the false-merge rate stays under 5%, then one step stricter. A run without labels uses it as it is. Here, with this file's truth: what each weight would accept.

In [9]:
```
with timed(f"O(C·G) = O({len(scores):,} pairs · 25 weights)"):
    grid = np.arange(-4.0, 21.0)
    curve = pd.DataFrame({"weight": grid,
                          "recall": [(scores["same"] & (w >= x)).sum() / len(wanted) for x in grid],
                          "false_pairs": [(~scores["same"] & (w >= x)).sum() for x in grid]})

fig, (a, b) = plt.subplots(1, 2, figsize=(10, 3.2))
a.plot(curve["weight"], curve["recall"], color=BLUE)
a.set(title="Share of the true pairs the rules left apart, accepted", xlabel="threshold (bits)", ylim=(0, 1))
a.yaxis.set_major_formatter(PercentFormatter(1))
b.plot(curve["weight"], curve["false_pairs"], color=ORANGE)
```

```
b.set(title="Pairs of different entities accepted", xlabel="threshold (bits)")
for ax in (a, b):
    ax.axvline(threshold, color=INK2, linewidth=1)
finish(fig)
```

```
O(C·G) = O(18,587 pairs · 25 weights) · 0.00s
```



10 · Cluster and evaluate

The accepted pairs and the rules' pairs are clustered together (connected components, in SQL). Gold gives each record its cluster and what resolved it: a rule, or the model. Then, with the truth: pairwise recall with its 95% interval, false merges, and the upper bound of the false-merge rate. Below: cluster sizes against true entity sizes, and the largest cluster. O(C + (N + E)·d).

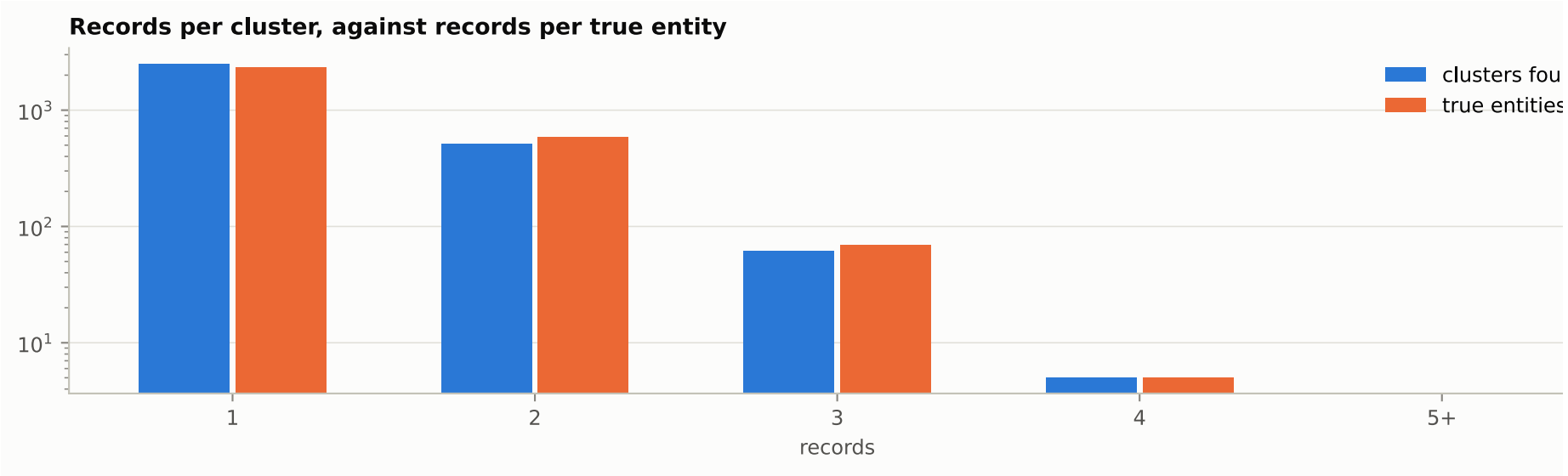
In [10]:
```
run.cluster()
done(run)
shutil.rmtree(DEMO / "run", ignore_errors=True)
run.write(DEMO / "run")
done(run)
gold = pd.read_parquet(DEMO / "run" / "gold.parquet").set_index(UID)
m = cluster_metrics(gold["cluster_id"], entity.reindex(gold.index))
print(f"recall {m['recall']:.1%} ({m['recall_lo']:.1%}–{m['recall_hi']:.1%}) · false merges {m['FP']} "
```

```
      f"(rate at most {m['fp_rate_hi']:.2%}) · precision {m['precision']:.2%}")
display(gold["resolved_by"].fillna("(alone)").value_counts().rename_axis("resolved by").to_frame("records"))

sizes = lambda ids: ids.value_counts().clip(upper=5).value_counts().reindex(range(1, 6), fill_value=0)
fig, ax = plt.subplots(figsize=(10, 3))
x = np.arange(1, 6)
ax.bar(x - 0.16, sizes(gold["cluster_id"]), width=0.3, color=BLUE, label="clusters found")
ax.bar(x + 0.16, sizes(entity), width=0.3, color=ORANGE, label="true entities")
ax.set_xticks(x, ["1", "2", "3", "4", "5+"])
ax.set(title="Records per cluster, against records per true entity", xlabel="records", yscale="log")
ax.grid(axis="x", visible=False)
ax.legend()
finish(fig)
largest = gold["cluster_id"].value_counts().index[0]
display(std.loc[gold.index[gold["cluster_id"] == largest], show].assign(entity=entity))
```

```
cluster: O(C + (N + E)·d) = O(18,587 + (3,747 + 145)·d) · 0.02s
write: O(N + C) = O(3,747 + 145) · 0.01s
recall 88.0% (85.6%–90.0%) · false merges 0 (rate at most 0.41%) · precision 100.00%
```

resolved by	records
(alone)	2509
X1_CIF_PERSON_NAME	284
X3_CIF_PHONE	266
fs	237
X4_PERSON_NAME_ADDRCD_PHONE	139
F1_NAME~_ADDRCD_PHONE	86
F2_NAME~_ADDRESS~_PHONE	60
X2_CIF_COMPANY_NAME	43
F3_NAME~_CIF	41
F7_GIVEN_ADDRCD_PHONE	39
X5_COMPANY_NAME_ADDRCD_PHONE	17
F4_NAME_ADDRCD_PHONE~	14
F5_COMPANY~_PHONE_POSTAL	9
F6_COMPANY~_ADDRCD_PHONE	3



CONT_ID	full_name	company_name	address	phone	customer_id	entity
0000000025	NaN	NaN	埼玉県南埼玉郡宮代町宮代1-8-26	4239	2381767751454	E001888
0000001250	サダナガハルキ	NaN	埼玉県南埼玉郡宮代町宮代1-8-26	4239	8371934185440	E001888
0000002123	サダナガハルキ	NaN	埼玉県南埼玉郡宮代町宮代1-8-26	4239	8371934185440	E001888
0000003284	サダナガハルキ	NaN	埼玉県南埼玉郡宮代町宮代1-8-26	4239	8371934185440	E001888

11 · Increment

New records only. An increment joins the batch against all the records, rule by rule and generator by generator, scores the pairs that touch it, and merges into the stored clusters, each existing cluster one node. So its cost grows with the batch, not with the file. Here: a full run on 90% of the records with the model fitted above, then the other 10% as an increment, compared with the full run on all of them in section 10. At this size the u sample inside `score` (a fixed S pairs) costs the most in both; the steps that follow the data (rules, candidates, cluster) follow the batch.

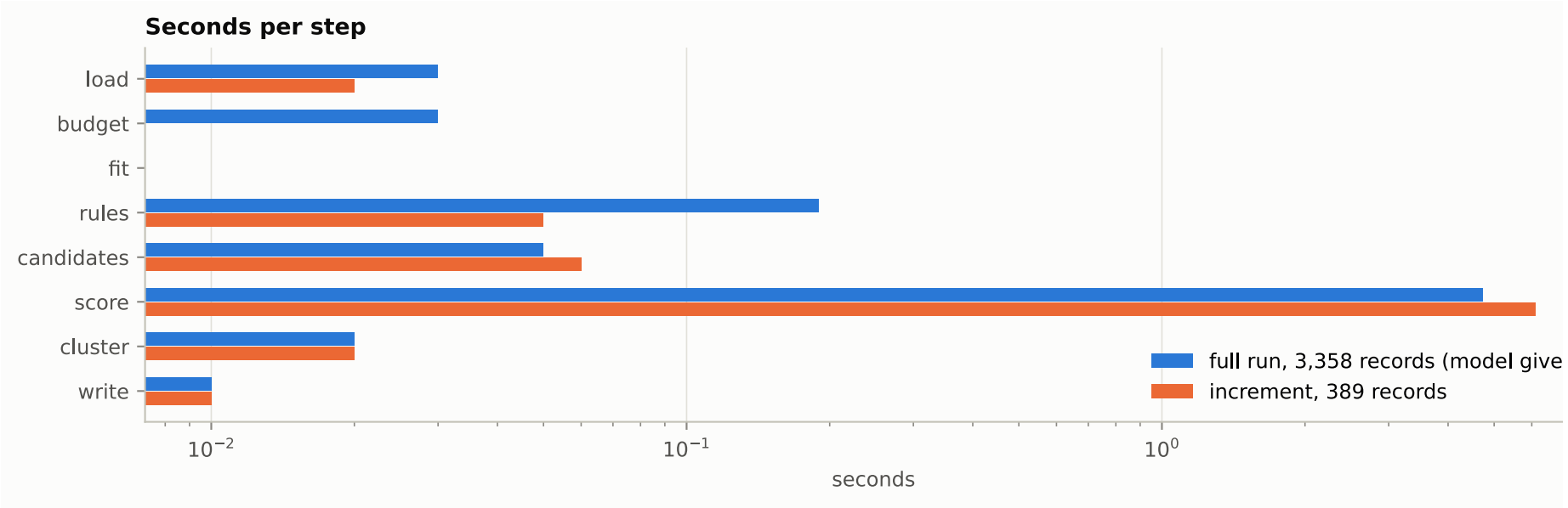
In [11]:
```
batch = np.random.default_rng(0).random(len(g.df)) < 0.1
g.df[~batch].to_parquet(DEMO / "first_90.parquet", index=False)
g.df[batch].to_parquet(DEMO / "new_10.parquet", index=False)
shutil.rmtree(DEMO / "increment", ignore_errors=True)
SCORED_U.clear()  # u of exactly these records is cached from section 7; a new batch makes a new set, so measure i
```

```
t again
base = Run(FINAL, S, model=model).full(DEMO / "first_90.parquet", DEMO / "increment")
inc = Run(FINAL, S).increment(DEMO / "new_10.parquet", DEMO / "increment")
for s in inc.steps:
    print(f"increment {s['step']}: {s['big_o']} = {s['filled']} · {s['seconds']:.2f}s")

after = pd.read_parquet(DEMO / "increment" / "gold.parquet").set_index(UID)["cluster_id"]
new_ids = set(g.df.loc[batch, UID])
old_in = after[~after.index.isin(new_ids)].unique()
joined = after[after.index.isin(new_ids) & after.isin(old_in)]
truth = predicted_pairs(entity)
by_inc, by_full = predicted_pairs(after), predicted_pairs(gold["cluster_id"])
print(f"{batch.sum():,} new records · {len(joined):,} joined an existing cluster · pairs only in the increment: "
      f"{len((by_inc - by_full) & truth)} true, {len((by_inc - by_full) - truth)} false · only in the full run: "
      f"{len((by_full - by_inc) & truth)} true, {len((by_full - by_inc) - truth)} false")

steps = ["load", "budget", "fit", "rules", "candidates", "score", "cluster", "write"]
seconds = lambda r: pd.Series({s["step"]: s["seconds"] for s in r.steps}).reindex(steps, fill_value=0)
fig, ax = plt.subplots(figsize=(10, 3.2))
y = np.arange(len(steps))[::-1]
ax.barh(y + 0.16, seconds(base), height=0.3, color=BLUE, label=f"full run, {(~batch).sum():,} records (model give
n)")
ax.barh(y - 0.16, seconds(inc), height=0.3, color=ORANGE, label=f"increment, {batch.sum():,} records")
ax.set_yticks(y, steps)
ax.set(title="Seconds per step", xlabel="seconds", xscale="log")
ax.grid(axis="y", visible=False)
ax.legend(loc="lower right")
finish(fig)
```

```
increment load: O(N·L) = O(389·L) · 0.02s
increment rules: O(|Δ|·Σ b + scan) = O(389 new records) · 0.05s
increment candidates: O(|Δ|·Σ b + scan) = 3,508 pairs touching the batch · 0.06s
increment score: O(N + S·K) + O(C_Δ·K·L²) = O(3,747 + 10,000,000·5) + O(3,508·5·L²) · 6.12s
increment cluster: O(C_Δ + touched clusters) = 29 accepted · 0.02s
increment write: O(N + C) = O(3,747 + 29) · 0.01s
389 new records · 120 joined an existing cluster · pairs only in the increment: 3 true, 0 false · only in the full r
un: 0 true, 0 false
```



12 · How the configuration was chosen

`scripts/choose.py` starts from `config/pipeline.toml` and tries the changes in `config/experiments.py` in order, each on top of the choice so far, on labelled synthetic data. A change is adopted by the decision rule: "better" needs a significant paired gain on the selection set; "cheaper" and "safer", no true pair lost; "chosen" is the owner's call, where no false pair may be added. On validation, all but "chosen" must lose no true pair. Test is scored once, at the end. Its tables are in `reports/choose.md`; read here, not recomputed.

In [12]:
```
text = (S.root / "reports" / "choose.md").read_text(encoding="utf-8")
for stage in ("## Stage 1: rules", "## Stage 2: Fellegi–Sunter"):
    decisions = table_after(text.split(stage, 1)[1], "### Decisions")
    print(stage[3:])
    display(decisions.iloc[:, :8])
print(next(line for line in (S.root / "config" / "final.toml").read_text(encoding="utf-8").splitlines() if line.st
artswith("# Adopted")))
```

```
Stage 1: rules
```

	change	kind	adopted	train_TP_gains	train_TP_losses	train_p	train_FP_added	train_FP_removed
0	fold_levenshtein	better	True	15	0	0.0001	0	0
1	fold_exact	cheaper	False	0	108	0.0	0	0
2	optuna_comparators	better	True	19	1	0.0	0	0
3	f2_address_exact	cheaper	False	0	8	0.0078	0	0
4	f2_address_prefix	cheaper	True	0	0	1.0	0	0

```
Stage 2: Fellegi–Sunter
```

	change	kind	adopted	cv_TP_gains	cv_TP_losses	cv_p	cv_FP_added	cv_FP_removed
0	no_term_frequency	better	True	15	0	0.0001	0	0
1	exact_keys_only	cheaper	False	0	3	0.25	0	0
2	optuna_suggestion	better	True	172	0	0.0	0	0
3	u_from_1e6_pairs	cheaper	False	9	26	0.006	0	0
4	compound_phone_keys	cheaper	True	0	0	1.0	0	0
5	sorted_names_for_tfidf	cheaper	True	1	0	1.0	0	0
6	name_keys	safer	True	1	0	1.0	0	0
7	bounded_address_keys	cheaper	True	0	0	1.0	0	0
8	no_sorted_windows	cheaper	False	0	3	0.25	0	0
9	bridge_rule_clusters	better	True	12	0	0.0005	0	0
10	u_from_scored_file	chosen	True	63	0	0.0	0	0
11	u_exact_by_counts	chosen	True	0	122	0.0	0	0

```
# Adopted: rules fold_levenshtein, optuna_comparators, f2_address_prefix; Fellegi–Sunter no_term_frequency, optuna_suggestion, compound_phone_keys, sorted_names_for_tfidf, name_keys, bounded_address_keys, bridge_rule_clusters, u_from_scored_file, u_exact_by_counts.
```

13 · Cost as the file grows

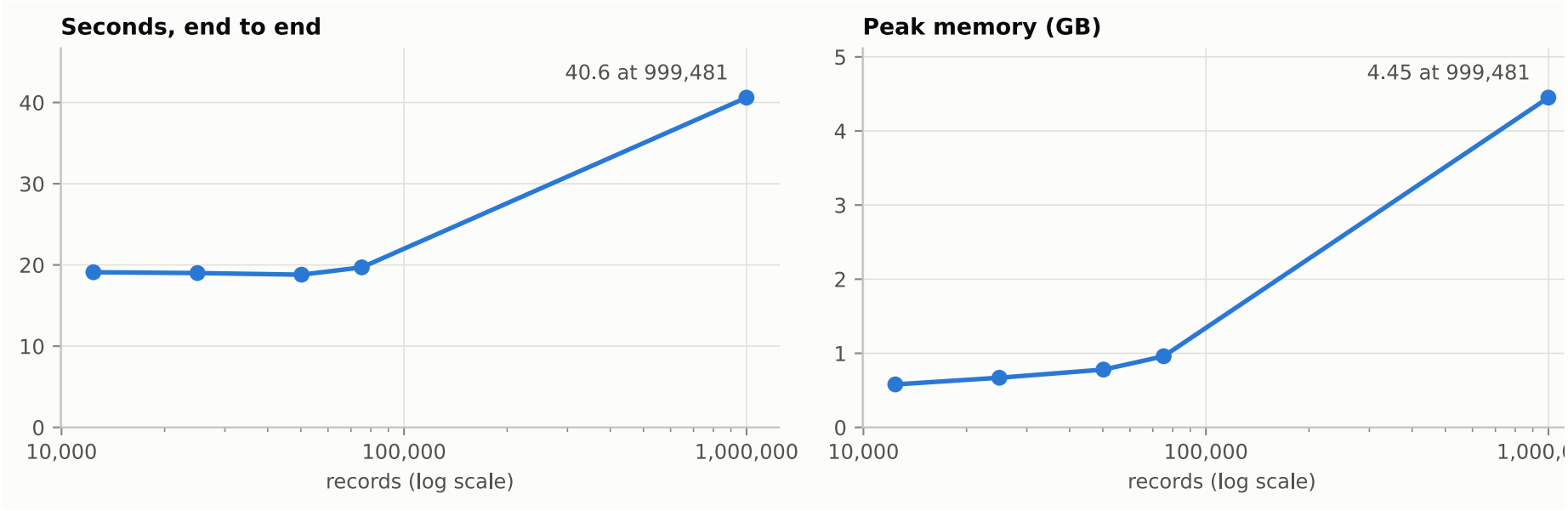
Every step's Big-O, for this full run and for the increment, with the numbers filled in. A full run is O(N·L + Σ P + C·K·L² + (N + E)·d), plus the fit and the u sample, whose sizes are fixed. When every key's pairs per record stay flat, that is linear in N. An increment of Δ records costs O(|Δ|·(L + Σ b) + C_Δ·K·L²) plus scans of the stored records. Below, measured by `scripts/benchmark.py` (`reports/benchmark.md`): each file resolved in a process of its own, up to a million records.

In [13]:
```
full_steps = pd.DataFrame(run.steps)[["step", "big_o", "filled", "seconds", "peak_memory_gb"]]
inc_steps = pd.DataFrame(inc.steps)[["step", "big_o", "filled", "seconds"]]
display(full_steps, inc_steps)
```

```
curve = table_after((S.root / "reports" / "benchmark.md").read_text(encoding="utf-8"), "## Cost")
curve = curve[pd.to_numeric(curve["seconds"], errors="coerce").notna()].astype({"records": int, "seconds": float,
"peak_memory_gb": float})
fig, (a, b) = plt.subplots(1, 2, figsize=(10, 3.2))
for ax, col, title in ((a, "seconds", "Seconds, end to end"), (b, "peak_memory_gb", "Peak memory (GB)")):
    ax.plot(curve["records"], curve[col], color=BLUE, marker="o", markersize=6)
    ax.set(title=title, xlabel="records (log scale)", xscale="log", ylim=(0, curve[col].max() * 1.15))
    ax.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x:,.0f}"))
    last = curve.iloc[-1]
    ax.annotate(f"{last[col]:g} at {last['records']:,}", (last["records"], last[col]), textcoords="offset points",
                xytext=(-8, 8), ha="right", color=INK2)
finish(fig)
```

	step	big_o	filled	seconds	peak_memory_gb
0	load	O(N·L)	O(3,747·L)	0.04	0.44
1	budget	O(N)	O(3,747) per rule and generator	0.03	0.45
2	rules	O(N + Σ P_r·L²) + O((N + E)·d)	O(3,747 + 564 pairs)	0.23	0.47
3	candidates	O(N·L + Σ P_g)	O(3,747·L + 19,357)	0.06	0.49
4	fit	O(n + S·K + P_em·K)	n = 3,747	6.24	0.65
5	score	O(N + S·K) + O(C·K·L²)	O(3,747 + 10,000,000·5) + O(18,587·5·L²)	5.66	0.65
6	cluster	O(C + (N + E)·d)	O(18,587 + (3,747 + 145)·d)	0.02	0.66
7	write	O(N + C)	O(3,747 + 145)	0.01	0.66

	step	big_o	filled	seconds
0	load	O(N·L)	O(389·L)	0.02
1	rules	O(|Δ|·Σ b + scan)	O(389 new records)	0.05
2	candidates	O(|Δ|·Σ b + scan)	3,508 pairs touching the batch	0.06
3	score	O(N + S·K) + O(C_Δ·K·L²)	O(3,747 + 10,000,000·5) + O(3,508·5·L²)	6.12
4	cluster	O(C_Δ + touched clusters)	29 accepted	0.02
5	write	O(N + C)	O(3,747 + 29)	0.01


Seconds, end to end
40.6 at 999,481
40
30
20
10
0
10,000
100,000
1,000,000
records (log scale)
Peak memory (GB)
4.45 at 999,481
5
4
3
2
1
0
10,000
100,000
records (log scale)